

PYTHON BASICS

180 Python Interview Questions with Trainer-Style Answers

Interview preparation • Fundamentals • Python 3

1. What is Python?

Answer: Python is a high-level, general-purpose programming language known for readable syntax. It is widely used in web development, automation, data analysis, AI/ML, scripting, and software development.

2. What are the key features of Python?

Answer: Key features include simple syntax, high-level abstractions, dynamic typing, automatic memory management, a large standard library, portability, object-oriented and multi-paradigm support, and a large ecosystem.

3. Why is Python called an interpreted language?

Answer: Python source code is normally compiled to bytecode and then executed by the Python virtual machine. It is commonly called interpreted because the bytecode is executed by a runtime rather than being directly compiled into native machine code beforehand.

4. What is the difference between compiled and interpreted languages?

Answer: A compiled language traditionally translates the whole program into native machine code before execution. An interpreted language executes source or an intermediate representation through a runtime. Modern languages often combine both approaches.

5. Why is Python platform independent?

Answer: Python is platform independent because the same Python source/bytecode can run on different operating systems when a compatible Python implementation is installed.

6. What makes Python dynamically typed?

Answer: Python is dynamically typed because names are associated with objects at runtime rather than having fixed declared types. The same name can refer to objects of different types at different times.

7. What is bytecode in Python?

Answer: Bytecode is an intermediate, platform-independent representation of Python code. It is designed for execution by a Python virtual machine.

8. What is PVM (Python Virtual Machine)?

Answer: PVM stands for Python Virtual Machine. It is the execution engine that interprets Python bytecode and performs the operations represented by that bytecode.

9. What is a .pyc file?

Answer: A .pyc file contains compiled Python bytecode for a module. Python may create it to speed up later imports.

10. What is the purpose of the __pycache__ folder?

Answer: __pycache__ is a directory where Python stores cached bytecode files, usually with .pyc extensions, for imported modules.

11. How does Python execute a program internally?

Answer: Typically: source code → tokenization/parsing → AST → bytecode compilation → bytecode execution by the PVM. Imports may also create cached .pyc files.

12. What is source code?

Answer: Source code is the human-readable program written by a programmer in a programming language such as Python.

13. What is machine code?

Answer: Machine code is processor-specific binary instructions that a CPU can execute directly.

14. What is compilation in Python?

Answer: In Python, compilation usually means translating source code into bytecode. This is different from compiling directly to native CPU machine code.

15. What is parsing in Python?

Answer: Parsing is the process of analyzing source text according to Python's grammar and determining its syntactic structure.

16. What is a parse tree?

Answer: A parse tree is a tree representation of the complete syntactic structure of source code, including many grammar details.

17. What is an Abstract Syntax Tree (AST)?

Answer: An Abstract Syntax Tree (AST) is a simplified tree representation of the meaningful syntactic structure of a program. It focuses on constructs such as expressions, statements, functions, and operators.

18. Difference between parse tree and AST?

Answer: A parse tree preserves detailed grammar structure, while an AST removes unnecessary syntactic details and represents the program's logical structure more compactly.

19. What role does AST play in execution?

Answer: The AST is an important intermediate representation used during compilation. Python analyzes it and compiles it into bytecode for execution.

20. Is Python fully interpreted? Explain.

Answer: No. Python is not purely interpreted. In common implementations such as CPython, source code is first compiled to bytecode and that bytecode is then executed by the virtual machine.

21. What is a variable?

Answer: A variable is a name that refers to an object in memory.

22. Why is there no need to declare variable types?

Answer: Python determines the type from the object assigned to a name, so explicit type declarations are generally unnecessary.

23. What is dynamic typing?

Answer: Dynamic typing means type checking and binding happen at runtime. A name can refer to different types of objects during program execution.

24. Can a variable change its type?

Answer: Yes. For example, `x = 10` followed later by `x = 'hello'` makes `x` refer to an integer and then a string.

25. What are built-in data types in Python?

Answer: Common built-in types include `int`, `float`, `complex`, `bool`, `str`, `list`, `tuple`, `set`, `frozenset`, `dict`, `range`, `bytes`, `bytearray`, `memoryview`, and `NoneType`.

26. What is the None type?

Answer: NoneType is the type of the special value None. None represents the absence of a value and is commonly used for missing results or default states.

27. What is mutable and immutable?

Answer: Mutable objects can be changed after creation; immutable objects cannot. Lists and dictionaries are mutable, while integers, strings, tuples, and frozensets are immutable.

28. Why are strings immutable?

Answer: Strings are immutable so their contents cannot be changed in place. Operations that appear to modify a string actually create a new string object.

29. What is type() function?

Answer: type() returns the type/class of an object. Example: type(10) returns <class 'int'>.

30. What is isinstance()?

Answer: isinstance(obj, cls) checks whether an object is an instance of a specified class or compatible subclass. It returns True or False.

31. What is an identifier?

Answer: An identifier is a name used for variables, functions, classes, modules, and other objects.

32. What are the rules for naming identifiers?

Answer: Identifiers may contain letters, digits, and underscores; they cannot start with a digit; they are case-sensitive; and keywords cannot be used as ordinary identifiers.

33. Can an identifier start with a number?

Answer: No. An identifier cannot begin with a digit. For example, 2name is invalid.

34. What is the difference between identifier and keyword?

Answer: An identifier is a programmer-defined name. A keyword is a reserved word with special meaning in Python, such as if, class, and return.

35. Is Python case-sensitive?

Answer: Yes. Python is case-sensitive, so Name, name, and NAME are different identifiers.

36. What is multiple assignment?

Answer: Multiple assignment assigns multiple values to multiple names in one statement, e.g. a, b = 10, 20.

37. What is chain assignment?

Answer: Chain assignment assigns one value to several names, e.g. a = b = c = 0.

38. How do you swap two variables?

Answer: Use tuple unpacking: a, b = b, a.

39. What is object identity?

Answer: Object identity indicates whether two references point to the exact same object. It can be tested with the is operator.

40. What does `id()` function do?

Answer: `id()` returns an integer that identifies an object for its lifetime. It is useful for identity-related debugging, not for comparing values.

41. What is `input()`?

Answer: `input()` reads a line of text from standard input and returns it as a string.

42. Why does `input()` always return a string?

Answer: Input is text by default because the function reads characters. Convert it explicitly when another type is required, such as `int(input())`.

43. What is `print()`?

Answer: `print()` writes objects to standard output, converting them to a human-readable representation.

44. How do you take integer input?

Answer: Use `int(input())`, for example: `age = int(input('Enter age: '))`.

45. How do you take multiple inputs in one line?

Answer: Use unpacking with `split()`: `a, b = input().split()`. For numbers: `a, b = map(int, input().split())`.

46. What are `sep` and `end` parameters?

Answer: `sep` controls the separator between multiple arguments of `print()`; `end` controls what is printed after the final argument. Defaults are `sep=' '` and `end='\n'`.

47. What is type casting?

Answer: Type casting means converting a value from one data type to another, such as `int('10')` or `float(5)`.

48. What is implicit conversion?

Answer: Implicit conversion is automatic conversion performed by Python in compatible numeric operations, such as converting an `int` to `float` in `5 + 2.5`.

49. What is explicit conversion?

Answer: Explicit conversion is conversion requested by the programmer using functions such as `int()`, `float()`, `str()`, or `bool()`.

50. Can every string be converted to `int`?

Answer: No. Only strings representing valid integer syntax can be converted. `int('123')` works, but `int('12.5')` raises `ValueError`.

51. What happens when `float` is converted to `int`?

Answer: Converting a `float` to `int` removes the fractional part toward zero. `int(7.9)` is 7 and `int(-7.9)` is -7.

52. What error occurs when conversion fails?

Answer: Usually `ValueError` occurs when the value has the right general type but an invalid format, e.g. `int('abc')`. `TypeError` can occur when the object type is unsupported.

53. What is `int()`?

Answer: `int()` converts suitable values to integers, e.g. `int('42') → 42` and `int(3.8) → 3`.

54. What is `float()`?

Answer: `float()` converts suitable values to floating-point numbers, e.g. `float('3.14') → 3.14`.

55. What is str()?

Answer: str() produces a string representation of an object.

56. What is bool()?

Answer: bool() converts a value to True or False according to Python's truth-value rules.

57. What is string formatting?

Answer: String formatting means inserting values into strings in a controlled readable form. Common methods include f-strings, format(), and older %-formatting.

58. What are f-strings?

Answer: F-strings are formatted string literals prefixed with f. Example: name='Aman'; f'Hello {name}' produces 'Hello Aman'. They can also contain expressions.

59. Difference between format() and f-strings?

Answer: f-strings are generally more concise and readable for current Python code. format() uses placeholders and method calls, and can be useful for reusable formatting patterns.

60. How do you print variables inside strings?

Answer: Use an f-string: name='Deepak'; age=20; print(f'{name} is {age} years old').

61. What is the difference between / and //?

Answer: / performs true division and normally returns a float. // performs floor division and returns the quotient rounded down toward negative infinity.

62. What is the use of % operator?

Answer: % returns the remainder after division. Example: 10 % 3 is 1.

63. What is the use of ** operator?

Answer: ** is the exponentiation operator. Example: 2 ** 3 is 8.

64. Why is ^ not a power operator?

Answer: ^ is the bitwise XOR operator, not exponentiation. Python uses ** for powers.

65. What are assignment operators?

Answer: Assignment operators include =, +=, -=, *=, /=, //=, %=, **=, &=, |=, ^=, >>=, and <<=.

66. Difference between = and ==?

Answer: = assigns a value to a name. == compares two values for equality and returns True or False.

67. What does += do internally?

Answer: a += b is generally equivalent to a = a + b, but augmented assignment can use an in-place operation when the object's type supports it.

68. What are relational operators?

Answer: Relational/comparison operators include ==, !=, <, >, <=, and >=.

69. Can relational operators be chained?

Answer: Yes. Python supports chained comparisons, such as a < b < c, which is roughly equivalent to a < b and b < c.

70. What is the output of `5 > 3 > 1`?

Answer: The output is True because `5 > 3` and `3 > 1` are both true.

71. What are logical operators?

Answer: Logical operators are and, or, and not. They combine or negate truth values and, for and/or, can return operands rather than only Boolean values.

72. Difference between and and `&`?

Answer: and is a logical short-circuit operator. & is bitwise AND and operates on integer bits or objects implementing the relevant operator.

73. Difference between or and `|`?

Answer: or is a logical short-circuit operator. | is bitwise OR and operates on bits or supported objects.

74. What does not operator do?

Answer: not reverses truthiness: not True is False and not False is True.

75. What are membership operators?

Answer: Membership operators are in and not in. They test whether a value is contained in a collection or supported object.

76. Difference between in and not in?

Answer: in returns True when the left operand is found in the right operand; not in returns True when it is absent.

77. What are identity operators?

Answer: Identity operators are is and is not. They test whether references point to the same object.

78. Difference between is and `==`?

Answer: == compares values for equality. is compares object identity. Use is commonly with None, e.g. `x is None`.

79. What are bitwise operators?

Answer: Bitwise operators work at the bit level: &, |, ^, ~, <<, and >>.

80. What is XOR operator?

Answer: XOR means exclusive OR. For each bit, the result is 1 when the corresponding bits differ and 0 when they are the same.

81. What is an if statement?

Answer: An if statement conditionally executes a block when its condition is truthy.

82. What is if-else?

Answer: if-else chooses between two blocks: one executes when the condition is true, the other when it is false.

83. What is elif ladder?

Answer: An elif ladder checks multiple conditions in sequence. The first true condition's block executes.

84. What is nested if?

Answer: A nested if is an if statement placed inside another conditional block.

85. What is a ternary operator?

Answer: Python's conditional expression is often called a ternary expression: `value_if_true if condition else value_if_false`.

86. What values are considered False in Python?

Answer: False, None, numeric zero values such as 0 and 0.0, empty strings, empty lists/tuples/sets/dictionaries, and other objects with false truth values are falsy.

87. What is truthiness?

Answer: Truthiness is Python's rule for deciding whether an object behaves as true or false in a Boolean context such as if or while.

88. What is match-case?

Answer: match-case is Python's structural pattern matching feature, introduced in Python 3.10. It can compare values and match structures/patterns.

89. How is match-case different from switch?

Answer: match-case is similar to a switch statement for simple value selection, but Python's pattern matching is more powerful because it can destructure and match patterns.

90. What is wildcard (_) in match-case?

Answer: In match-case, `_` is the wildcard pattern. It matches anything and is commonly used as a default case.

91. Can match-case handle multiple values?

Answer: Yes. For example: `case 1 | 2 | 3:` matches any of those values.

92. When should match-case be used?

Answer: Use match-case when the decision is naturally expressed as matching values or structural patterns, especially when it improves readability.

93. Can match-case match patterns?

Answer: Yes. It can match sequences, mappings, classes, literals, alternatives, and bind values using structural patterns.

94. What happens when no case matches?

Answer: If no case matches, execution simply continues after the match statement unless a wildcard case is provided.

95. Can match-case replace if-elif ladders?

Answer: Yes, for many situations. However, complex boolean conditions may still be clearer with if-elif.

96. What is a for loop?

Answer: A for loop iterates over the items of an iterable such as a list, string, tuple, or range.

97. What is a while loop?

Answer: A while loop repeatedly executes a block while its condition remains truthy.

98. Difference between for and while?

Answer: for is generally used to iterate over a known iterable; while is used when repetition depends on a condition that changes during execution.

99. What is range()?

Answer: range() produces an immutable sequence-like range of integers, commonly used for iteration.

100. Explain range(start, stop, step).

Answer: range(start, stop, step) starts at start, progresses by step, and stops before stop. Example: range(1, 6, 2) gives 1, 3, 5.

101. Can range have negative step?

Answer: Yes. Example range(5, 0, -1) gives 5, 4, 3, 2, 1.

102. What is an infinite loop?

Answer: An infinite loop is a loop whose condition never becomes false and which has no effective termination path.

103. How do you stop an infinite loop?

Answer: Use a correct terminating condition or break. During an accidental infinite loop, interrupt execution with the environment's interrupt command, commonly Ctrl+C in a terminal.

104. What is a nested loop?

Answer: A nested loop is a loop inside another loop. For each outer iteration, the inner loop may execute repeatedly.

105. What is loop control?

Answer: Loop control means controlling normal iteration using statements such as break, continue, and pass.

106. What is break?

Answer: break immediately terminates the nearest enclosing loop.

107. What is continue?

Answer: continue skips the rest of the current iteration and starts the next iteration.

108. What is pass?

Answer: pass is a no-operation statement used as a placeholder where Python requires a statement.

109. Difference between break and continue?

Answer: break exits the loop completely; continue skips only the current iteration.

110. What is for-else?

Answer: for-else has an else block that executes if the for loop finishes normally without encountering break.

111. What is while-else?

Answer: while-else works similarly: the else block runs when the while condition becomes false without a break.

112. When does loop else execute?

Answer: The loop's else executes when the loop terminates normally, not when it is terminated by break.

113. Does else execute after break?

Answer: No. If break terminates the loop, its else block is skipped.

114. What is loop nesting complexity?

Answer: With nested loops, work can multiply. For example, two independent loops nested over n items can result in $O(n^2)$ iterations; three can result in $O(n^3)$. Actual complexity depends on loop bounds.

115. How do you iterate through a string?

Answer: Use for character in string: `print(character)`. You can also use `enumerate()` when the index is needed.

116. What is a list?

Answer: A list is an ordered, mutable collection that can contain multiple objects, including duplicates and different types.

117. Why are lists mutable?

Answer: Lists are designed to support in-place changes such as `append`, `remove`, and item assignment. Their mutability is part of the list data model.

118. How do you create a list?

Answer: Use square brackets: `numbers = [1, 2, 3]`. An empty list is `[]`.

119. What is indexing in lists?

Answer: Indexing accesses an item by position, starting at index 0. For example, `a[0]` is the first item.

120. What is negative indexing?

Answer: Negative indexing counts from the end. `a[-1]` is the last item.

121. What is slicing?

Answer: Slicing extracts a range using `sequence[start:stop:step]`. The stop index is excluded.

122. Difference between `append()` and `extend()`?

Answer: `append(x)` adds `x` as one item. `extend(iterable)` adds each item from the iterable.

123. Difference between `append()` and `insert()`?

Answer: `append(x)` adds `x` to the end. `insert(index, x)` adds `x` at a specified position and shifts later elements.

124. Difference between `remove()` and `pop()`?

Answer: `remove(x)` removes the first matching value and raises `ValueError` if absent. `pop(index)` removes and returns an item, defaulting to the last item.

125. What does `clear()` do?

Answer: `clear()` removes all items from a list.

126. What does `copy()` do?

Answer: `copy()` creates a shallow copy of a list.

127. Difference between `sort()` and `sorted()`?

Answer: `sort()` sorts a list in place and returns `None`. `sorted()` returns a new sorted list and works with any iterable.

128. Difference between `reverse()` and `reversed()`?

Answer: `reverse()` reverses a list in place and returns `None`. `reversed()` returns an iterator that yields items in reverse order.

129. How do you find list length?

Answer: Use `len(list_name)`.

130. How do you find max and min values?

Answer: Use `max(iterable)` and `min(iterable)`, provided the elements can be meaningfully compared.

131. How do you count occurrences?

Answer: Use `list.count(value)` to count occurrences of a value.

132. How do you merge two lists?

Answer: Use `a + b` to create a new merged list, or `a.extend(b)` to modify `a` in place.

133. How do you remove duplicates?

Answer: Common approaches include converting to a set when order requirements allow it, or using a loop/dictionary to preserve order. `dict.fromkeys(list)` is a common order-preserving technique.

134. What is a nested list?

Answer: A nested list is a list containing other lists, e.g. `[[1, 2], [3, 4]]`.

135. How do you traverse a nested list?

Answer: Use nested loops, e.g. `for row in matrix: for item in row: print(item)`.

136. What is shallow copy?

Answer: A shallow copy creates a new outer object but keeps references to nested objects.

137. What is deep copy?

Answer: A deep copy recursively copies nested objects so changes to copied nested mutable objects do not affect the original.

138. Difference between shallow and deep copy?

Answer: Shallow copy shares nested object references; deep copy recursively duplicates them. `copy.copy()` makes a shallow copy and `copy.deepcopy()` makes a deep copy.

139. What happens in `a=b`?

Answer: `a = b` makes `a` and `b` refer to the same object. It does not create a new copy.

140. What happens in `a=b.copy()`?

Answer: `a = b.copy()` creates a new outer list, but nested objects are still shared because `list.copy()` is shallow.

141. What is a tuple?

Answer: A tuple is an ordered, immutable collection. It can contain duplicate values and objects of different types.

142. Why are tuples immutable?

Answer: Tuple immutability provides a fixed sequence structure and can make tuples suitable for safe fixed data and hashable keys when their contents are hashable.

143. How do you create a tuple?

Answer: Use parentheses or simply comma-separated values: `t = (1, 2, 3)`. Parentheses are often optional.

144. What is a single-element tuple?

Answer: A single-element tuple requires a trailing comma: `(5,)`.

145. Difference between (5) and (5,)?

Answer: (5) is simply the integer 5 inside parentheses. (5,) is a tuple containing one element.

146. What is tuple packing?

Answer: Tuple packing is collecting multiple values into one tuple, e.g. `t = 1, 2, 3`.

147. What is tuple unpacking?

Answer: Tuple unpacking assigns tuple elements to multiple names, e.g. `a, b = (10, 20)`.

148. What is extended unpacking?

Answer: Extended unpacking uses `*` to collect multiple remaining items: `a, *middle, b = [1, 2, 3, 4]`.

149. What is a nested tuple?

Answer: A nested tuple is a tuple containing another tuple, such as `((1, 2), (3, 4))`.

150. Can a tuple contain mutable objects?

Answer: Yes. A tuple can contain mutable objects such as lists. The tuple's references cannot be changed, but the referenced list can still be modified.

151. Advantages of tuple over list?

Answer: Tuples provide immutability, can be more memory-efficient, and are often slightly faster to iterate than equivalent lists. They can also be hashable when all contained elements are hashable.

152. Which is faster: list or tuple?

Answer: For common iteration and indexing, tuples are often slightly faster, although the difference depends on the operation and implementation. Choose based on semantics, not just speed.

153. Which consumes less memory?

Answer: Tuples generally consume less memory than equivalent lists because their structure is smaller and fixed-size.

154. When should tuple be preferred?

Answer: Prefer tuples for fixed collections, records, function returns, and data that should not be structurally modified.

155. Can a tuple be a dictionary key?

Answer: Yes, if every element of the tuple is hashable. A tuple containing a list cannot be a dictionary key because the list is unhashable.

156. What is the output of `print(True + True)`?

Answer: 2. In Python, `bool` is a subclass of `int`, so `True` behaves like 1 and `True + True` equals 2.

157. What is the output of `print(False + True)`?

Answer: 1. `False` behaves like 0 and `True` like 1.

158. What is the output of `print(bool(""))`?

Answer: `False`. An empty string is falsy.

159. What is the output of `print(bool('0'))`?

Answer: `True`. A non-empty string, even the string `'0'`, is truthy.

160. What is the output of `print(10 and 0 or 5)`?

Answer: 5. `and` returns the first falsy operand or the last operand; 10 and 0 gives 0, then 0 or 5 gives 5.

161. What is the output of `print(5 > 3 > 2)`?

Answer: True. Both comparisons, `5 > 3` and `3 > 2`, are true.

162. What is the output of `print(10/3)`?

Answer: 3.3333333333333335 on typical Python 3 implementations.

163. What is the output of `print(10//3)`?

Answer: 3.

164. What is the output of `print(10%3)`?

Answer: 1.

165. What is the output of `print(2**3**2)`?

Answer: 512. Exponentiation is right-associative: $2 ** (3 ** 2) = 2 ** 9 = 512$.

166. Why is Python slower than Java?

Answer: Python can be slower for CPU-bound code because common CPython execution uses bytecode and dynamic runtime operations, while Java commonly uses JVM JIT compilation to optimize hot code to native machine instructions. This is a general comparison, not an absolute rule.

167. Why is Python popular in AI?

Answer: Python has readable syntax, extensive scientific libraries, mature ML frameworks, rapid development, strong community support, and easy integration with data-processing tools.

168. Why does Python support multiple paradigms?

Answer: Python supports procedural, object-oriented, and functional programming styles. Its flexible object model and first-class functions allow different programming paradigms.

169. What happens when a syntax error occurs?

Answer: A syntax error is detected when Python cannot parse the source according to its grammar. Execution does not proceed for that code, and Python reports a `SyntaxError` with location information when possible.

170. What happens when an exception is not handled?

Answer: If an exception is not handled, it propagates up the call stack. If nothing handles it, Python terminates the program and prints a traceback describing the exception.

171. Why are lists mutable but tuples immutable?

Answer: Lists are mutable because they are designed for collections that can change. Tuples are immutable because they represent fixed sequences and provide useful properties such as hashability when their elements are hashable.

172. Why does `input()` return string?

Answer: `input()` reads user-entered text. Returning a string avoids guessing whether the user intended an integer, float, date, or other type; conversion is left to the programmer.

173. What happens when Python starts execution?

Answer: When Python starts a script, it initializes the interpreter, creates the execution environment, loads necessary modules, compiles the source to bytecode, and executes the top-level code. Exact internal details depend on the implementation.

174. How does memory management work in Python?

Answer: Python manages memory automatically. In CPython, objects live in a managed heap, references are tracked, and memory is reclaimed primarily through reference counting plus cyclic garbage collection.

175. What is garbage collection in Python?

Answer: Garbage collection is automatic reclamation of objects that are no longer reachable. CPython primarily uses reference counting and also has a cyclic garbage collector to handle reference cycles.

176. Difference list versus tuple versus set

Answer: List: ordered, mutable, duplicates allowed. Tuple: ordered, immutable, duplicates allowed. Set: unordered collection of unique hashable elements and mutable as a container. Choose based on whether order, mutability, or uniqueness matters.

177. What is empty set

Answer: An empty set is created with `set()`, not `{}`. `{}` creates an empty dictionary.

178. Can we use `sort()` with tuple why or why not.

Answer: No. tuple objects do not have a `sort()` method because tuples are immutable. Use `sorted(t)`, which returns a new list, or convert to a list, sort it, and create a tuple if needed.

179. What is tuple comprehension?? explain in detail.

Answer: Python has no dedicated tuple-comprehension syntax. An expression like `(x*x for x in range(5))` is a generator expression, not a tuple. To create a tuple, wrap it with `tuple()`: `tuple(x*x for x in range(5))`. A list comprehension `[x*x for x in range(5)]` creates a list.

180. What is `eval()`?? explain with proper examples.

Answer: `eval()` evaluates a string as a Python expression and returns its result. Example: `eval('2 + 3')` returns 5; `x=10; eval('x * 2')` returns 20. It can evaluate expressions involving names available in its namespace. IMPORTANT: never use `eval()` on untrusted user input because it can execute arbitrary Python expressions and create serious security risks. For safe conversion of simple literals, `ast.literal_eval()` is usually a better choice, e.g. `ast.literal_eval('[1, 2, 3]')`.